Scientific Visualization and Octopus Development
– Hands-On Session –



Michele Compostella & Markus Rampp

Max Planck Computing and Data Facility (MPCDF)

Tutorial Workshop on Scientific Visualization and Octopus Development, Hamburg 6-7.07.2016

Overview

Visit's Command Line Interface (CLI)

Introduction:

- What is VisIt's CLI?
- Advantages with respect to VisIt's GUI
- How to run VisIt's CLI
- How to script for VisIt's CLI: some basic commands

Visualization of Octopus data using scripts:

- *C2H4_2D.py* : simple, 2D plot of the proton-molecule scattering
- *C2H4_3D.py* : introducing camera movements and volume plots
- *azulene.py* : multiple plotting windows, queries and new expressions

In-depth analysis of the scripts:

- How and what to change in the scripts to better fit your needs

VisIt's CLI

VisIt's command line interface (or CLI) is a Python-based, alternative interface with respect to VisIt's graphical user interface (or GUI).



You type the instructions in a terminal instead of clicking a button

It includes a large set of instructions based on Python 2.7, together with several commands specific of the VisIt's distribution.



You can write small Python programs for the analysis of the data

Using VisIt's CLI you have access to the same operations (plots, operators, queries, etc.) available in the graphical user interface.



As powerful as the GUI, but with some advantages

VisIt's GUI vs VisIt's CLI

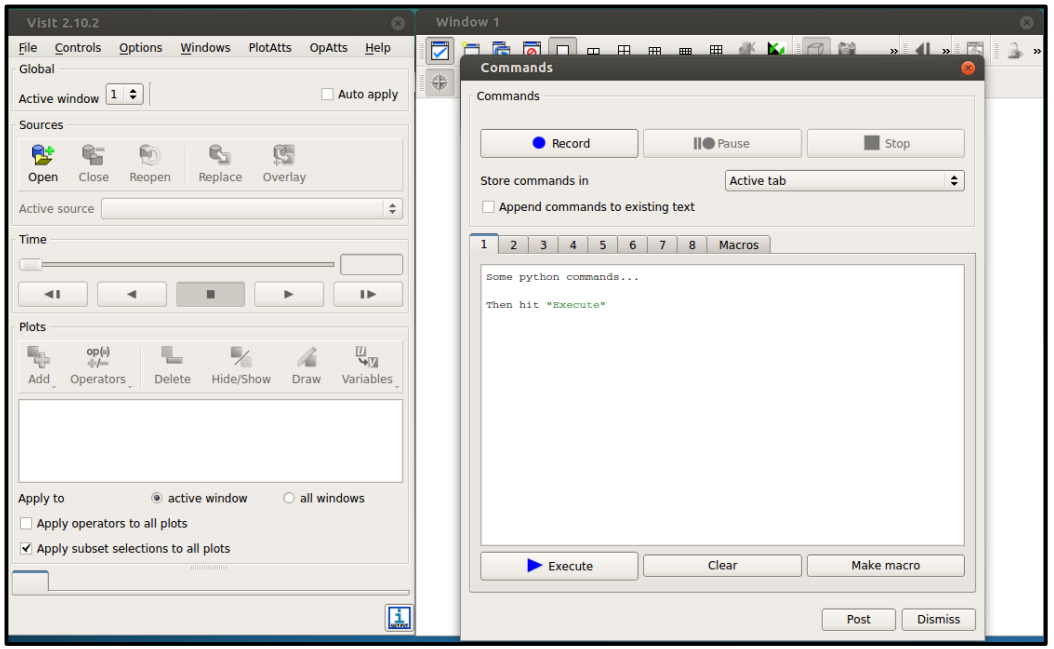
VisIt's GUI

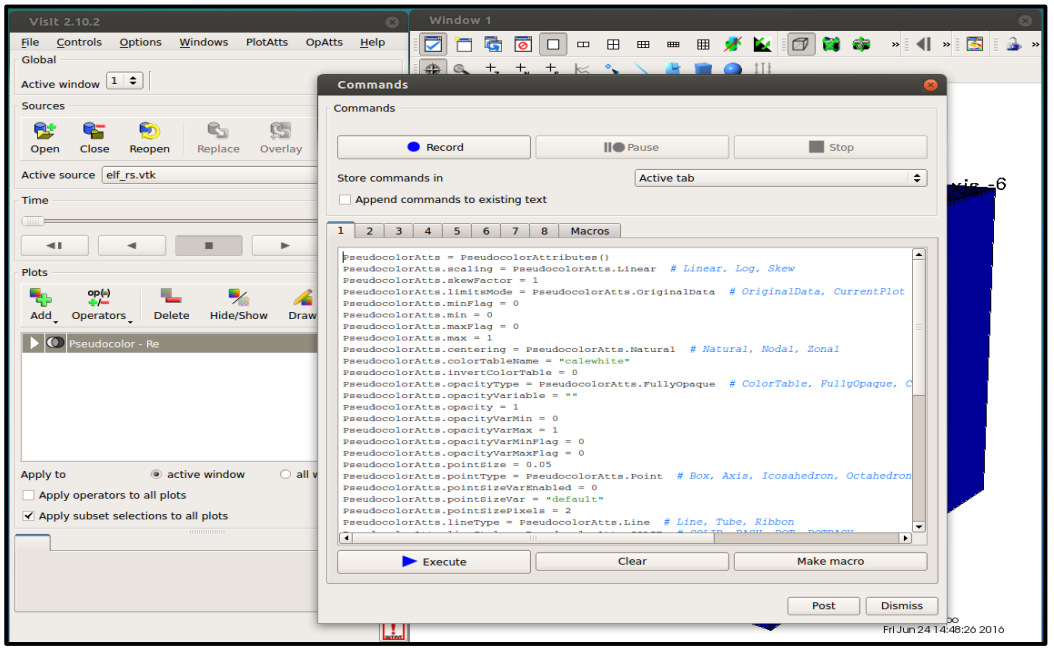
- Easy to use
- Quick overview of the dataset
- Useful for a single plot or image

- You can use the “Commands” window to submit Python code

VisIt's CLI

- Steeper learning curve
- Requires a complete set of instructions
- Useful for multiple tests on the same dataset, for large sets of data or for creating images with the same characteristics (e.g. movies)
- You can record from the GUI and use the code in a Python script for the CLI





Launching VisIt's CLI

There are 2 main ways to run VisIt's CLI:

- `visit -cli`

  for a version of VisIt's CLI with plotting windows. Useful when you want to interact with the plot and with the view.

- `visit -cli -nowin`

  for a version without plotting window. Useful when you run scripts and you want all the calculations to be done in background.

VisIt's CLI: basic commands

We will use the file "`elf_rs.vtk`" contained in the directory "`td.0000000/`":

`(1) OpenDatabase("elf_rs.vtk")`	• opens a database to access its variables
`(2) AddPlot("Pseudocolor", "Re")`	• selects the plot type and the variable name
`(3) Var1 = PseudocolorAttributes()`	• retrieves the attributes of the plot and saves them in the variable Var1 (you can print them)
`(4) SetPlotOptions(Var1)`	• sets the new values of the plot
`(5) AddOperator("Slice", 0)`	• adds an operator called "slice" to the current plot
`(6) Var2 = SliceAttributes()`	• retrieves the attributes of the operator
`(7) SetOperatorOptions(Var2)`	• sets the new values for the attributes of the operator
`(8) DrawPlots()`	• draws all the plots and applies all the operators
`(9) SaveWindow()`	• saves a copy of the plot on hard disk
`(10) DeleteAllPlots()`	• deletes all plots (both active and not)
`(11) CloseDatabase("elf_rs.vtk")`	• closes the selected database
`(12) quit()`	• quits from the CLI and closes VisIt

Let's try!

Scripting for VisIt's CLI

Problem:

- Complex plots require lots of instructions. Typing them can be tedious.
- Producing several images implies typing the same instructions multiple times.

Solution:

- Collect instructions into a single Python script that can be provided to VisIt when launched.

Advantages:

- Fast method to analyze several inputs, once the script is ready.
- Less prone to errors.
- Possibility to re-create a plot in the future.
- Possibility to share and create standardize images between collaborators.
- Produce the same plot for several different input files.
- Creation of frames for a movie.

How to run the scripts

How to run a script with VisIt's CLI:

With a plotting window

- `visit -cli -s script.py`

or without

- `visit -cli -nowin -s script.py`

Using a Python interpreter:

On an interactive node on Hydra@MPCDF:

```
> module load python27/python/2.7

> python2.7 script.py
```

But you must provide the location of the directory “site-packages” from your installation of VisIt.

On Hydra:

```
> $VISIT_HOME/current/linux-x86_64/lib/site-packages

> /hydra/u/system/SLES11/soft/visit/2.10/current/linux-
x86_64/lib/site-packages
```

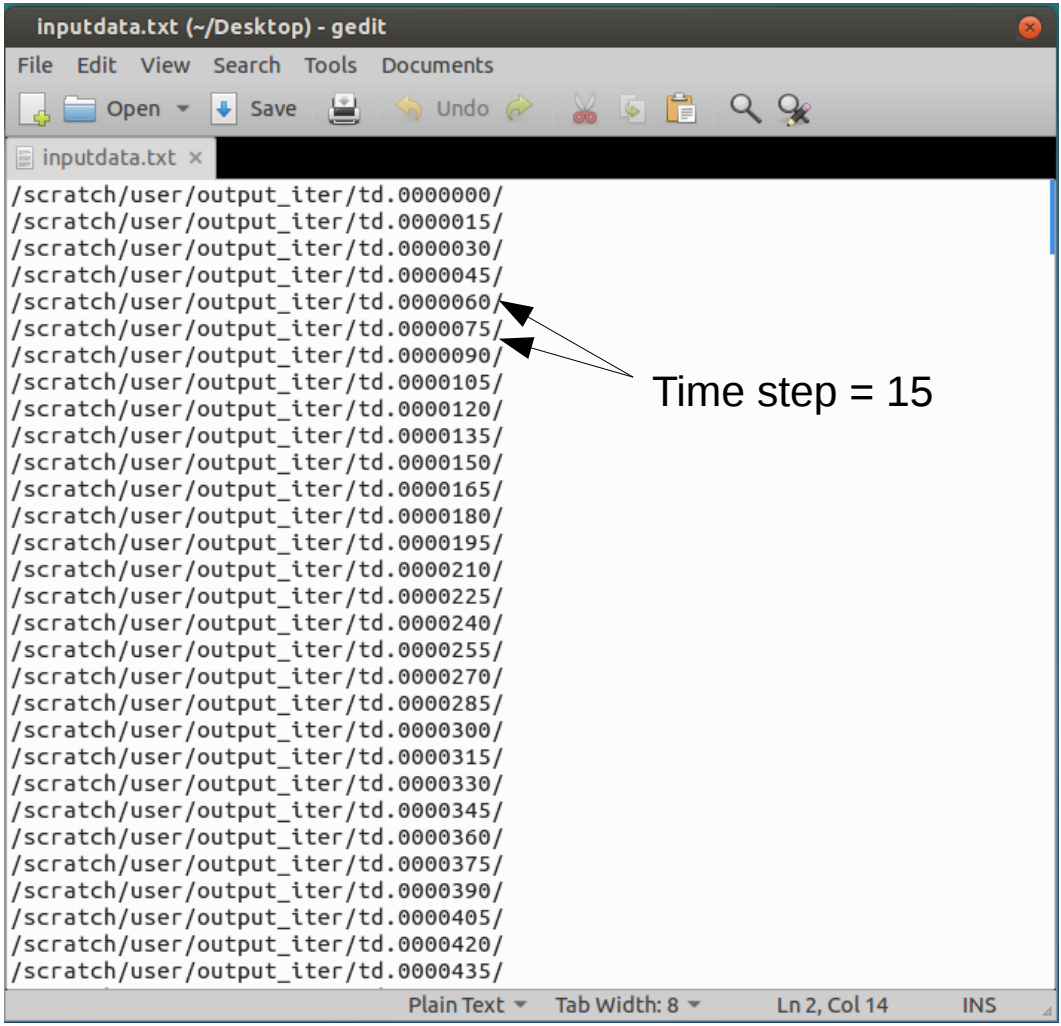
How to run the scripts

How to provide the location of the input files:

1) You use the default values hard-coded into the script. This option requires an “inputdata.txt” file containing the location of the inputs (see image). Useful if you want to modify the script or you have to work with the same set of data multiple times (e.g. for tests).

2) You explicitly provide the location of the “inputdata.txt”, where each line corresponds to a different input file. Useful when you have inputs in different paths or the time step between inputs is not constant.

3) This option does not require “inputdata.txt”. You must provide the location of the inputs via command line, the name of the first and last inputs and the time steps between inputs.



Once the location of the inputs is provided, several other options may be available:

- Output directory, plot limits, image name, movie options, etc.

How to run the scripts

The scripts use a VisIt's utility to create movies:

On Hydra you may want to load the audio/video codec library using

```
> module load ffmpeg
```

You also need to specify a speed between 1 and 100. This number select how many times each image will be used in the production of the movie frames (the output framerate is always 25 fps). You can also produce a movie using only old images (see the in-depth of the script).

Alternatively, you can use the ffmpeg command:

In a terminal, you can produce a movie using the following instruction

```
> ffmpeg -framerate 10 -start_number 0 -i mol2D%04d.png -c:v libx264
-vf "fps=25,format=yuv420p" mol2D_2D.mp4
```

where:

- framerate: is the number of images per second used as input
- start_number: is the number id of the first image
- mol2D%04d.png: specifies the name of the input images. %04d indicates 4 integer numbers
- fps=25: is the number of frames per second in the output (ffmpeg takes care of using the input images the correct number of times)
- mol2D_2D.mp4: is the name of the output

Description:

- Creates a set of 2-dimensional images of the scattering of a proton off a molecule of C_2H_4.
- Each image is a slice of the volume along the plane of the molecule.
- Each image is made of 3 plots: 2 pseudocolor plots (one for the electron localization function and one for the elements of the molecule) and 1 contour plot (electron localization function equal to 0.8).
- You can add the molecular bonds by modifying the variable “bonds” in the script.

Parameters:

- Plot limits:

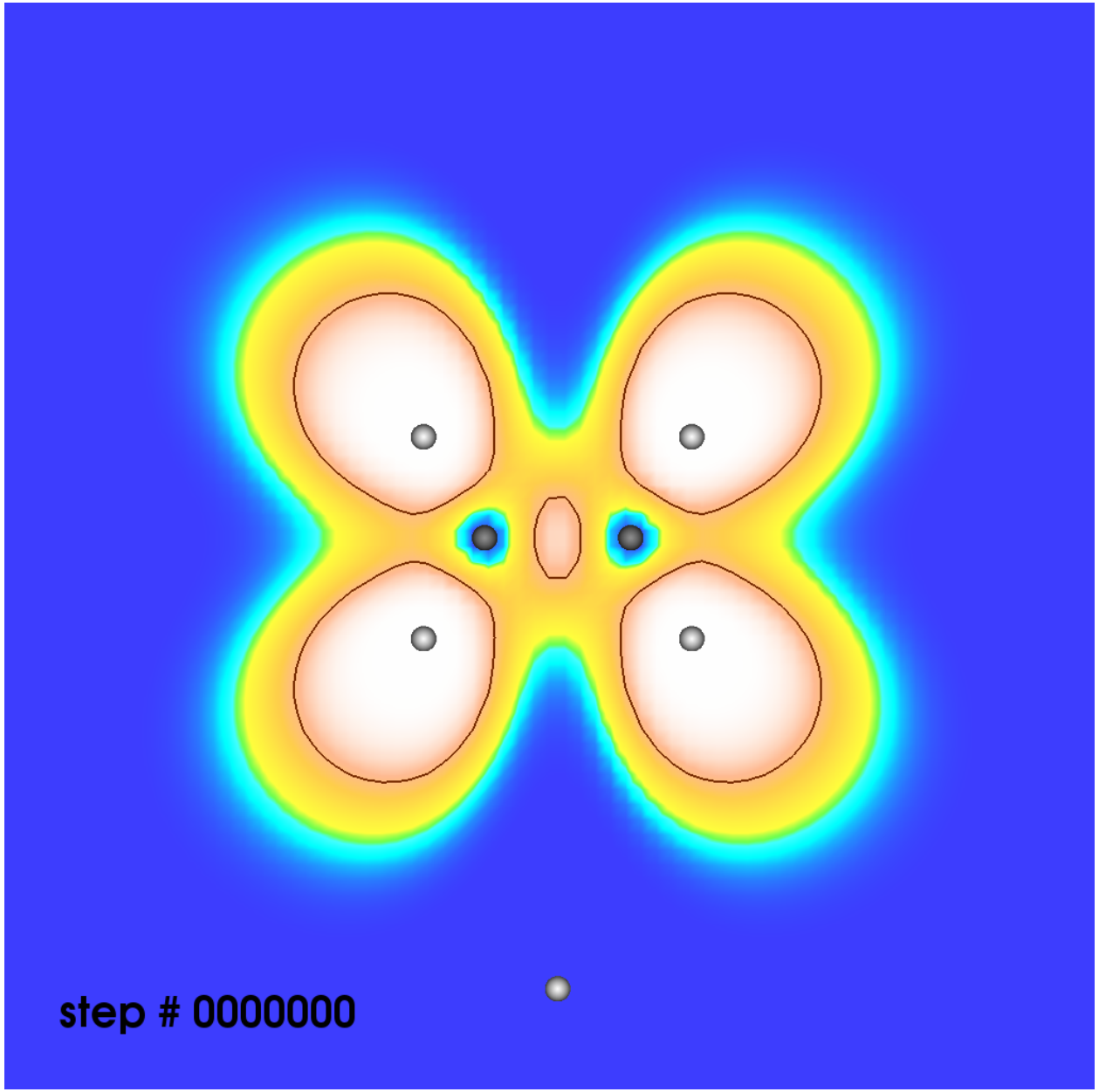
  The limits of the visible region in the image are provided as four different values separated by a single space

  ```
  xmin xmax ymin ymax
  ```

- Color table:

  The default color table is hard-coded into the script, but it can be changed to one of the several color tables provided by VisIt.

C2H4_2D.py



Let's try!

C2H4_3D.py

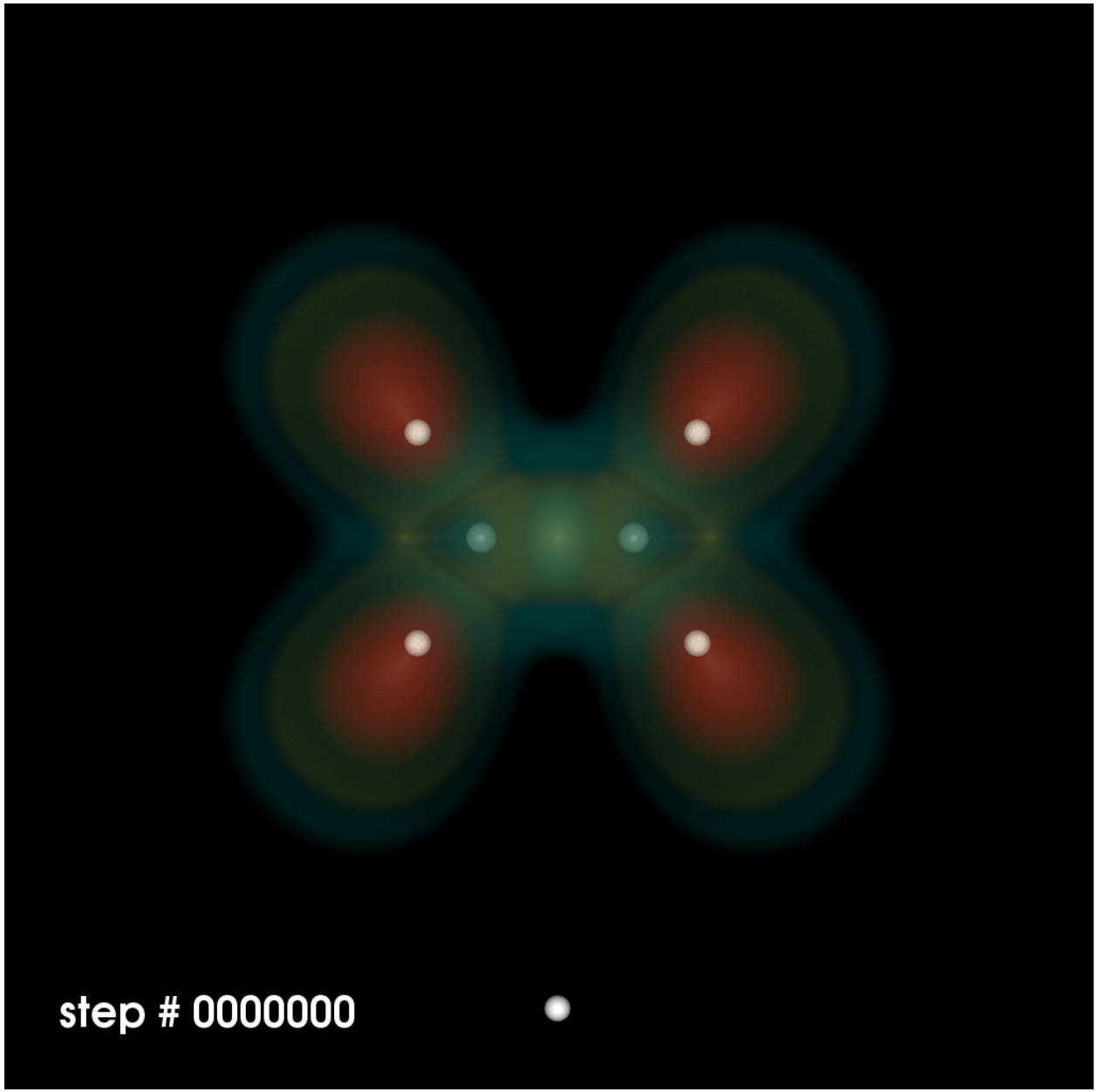
Description:

- Creates a set of 3-dimensional images of the scattering of a proton off a molecule of C_2H_4.
- Can choose between volume and contour plots.
- Introduces camera movements.
- You can add the molecular bonds by modifying the variable “bonds” in the script.

Parameters:

- Zoom: modifies apparent distance between the camera point and the molecule.
- Plot type:
  - Volume plot: volume rendering with 3D texturing.
  - Volume plot with ray casting.
  - Multi-contour plot using isosurfaces.
- Background grid: possibility to introduce a background grid.
- Camera: rotation of the camera around the molecule for the first input and/or during the scattering process.
- Color table: all the standard color tables provided by VisIt are available for 3D texturing and contour plots. The color table for the volume plot with ray casting can be modified in the script (but see in-depth explanation of *C2H4_3D.py* for this).

C2H4_3D.py



Let's try!

Description:

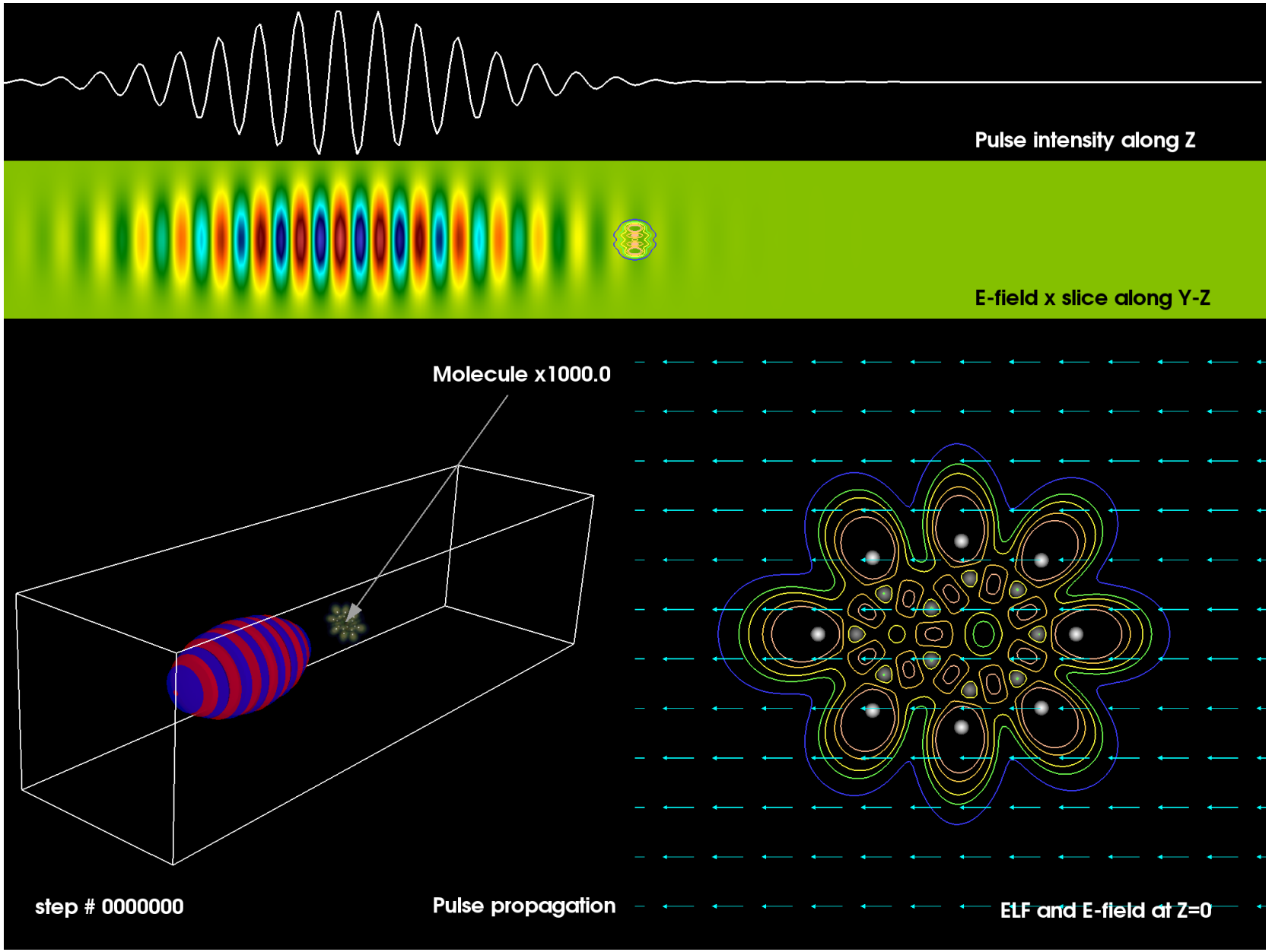
- Creates a mosaic of 4 images describing different aspects of the interaction of a laser pulse with a molecule of Azulene:
  - Intensity of the electric field along axis Z
  - Slice of the electric field along plane Y-Z and position of maxima/minima with respect to the molecule
  - 3-dimensional propagation of the laser pulse
  - Electron localization function isosurfaces, position of the elements of the molecule and vectors representing the electric field.
- You can add the molecular bonds by modifying the variable “bonds” in the script.

Parameter:

- Zoom: the typical scale of a laser pulse and a molecule can be very different. This parameter rescales the size of the molecule in order to plot both the laser pulse and the molecule on the same plot. If this parameter is larger than 1, the script creates 100 frames where the molecule is gradually rescaled to the final zoom.

Notes:

- Window 5: the script performs some of the calculations on a window indicated as “Window 5” that is created, manipulated and destroyed at each time step.


Pulse intensity along Z
E-field x slice along Y-Z
Molecule x1000.0
step # 0000000
Pulse propagation
ELF and E-field at Z=0

Let's try!

In-depth analysis of *C2H4_2D.py*



In-depth analysis of *C2H4_2D.py*

```
##############################################################################
## Copyright (C) 2016 M. Compostella, M. Rampp, H. Appel
##
## This program is free software; you can redistribute it and/or modify
## it under the terms of the GNU General Public License as published by
## the Free Software Foundation; either version 2, or (at your option)
## any later version.
##
## This program is distributed in the hope that it will be useful,
## but WITHOUT ANY WARRANTY; without even the implied warranty of
## MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
## GNU General Public License for more details.
##
## You should have received a copy of the GNU General Public License
## along with this program; if not, write to the Free Software
## Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA
## 02110-1301, USA.
##
##############################################################################
## General Notes:
## This is a python script for the visualization sotfware VisIt.
## Submit the script with "visit -cli -s script_name.py"
## or "visit -cli -nowin -s script.py" for a version without plotting windows
##
## Last update: 29.06.2016
## Tested on: VisIt 2.10 (patch 2)
## at: Max Planck Computing and Data Facility, Garching
##############################################################################
```

General Public License

Some general comments

In-depth analysis of *C2H4_2D.py*

List and description of the variables that can be changes

```
# This script creates a 2D plot of the scatter of a proton off a molecule of C2H4.


# Variables that you can set before running the script:
#
# inputdata     - File with the FULL name (ending with "/") of the directories containing the data that you
#                 want to plot. Each line is an input step for the plot.
#
# outdir        - Directory where the .png images are saved.
#
# elfvtk        - Name of the .vtk file containing the Octopus results. The script assumes a variable called 'Re' is
#                 present in this file.
#
# geometry      - Name of the file containing the geometry of the molecule (e.g. geometry.xyz).
#
# elfcube       - Name of the .cube file containing the Octopus results and the position of the elements of the molecule.
#                 The script assumes the variables named 'electron_density' and 'elements' are present in this file.
#
# bonds         - Set to True if you want to plot the bonds of the molecule. Note that bonds are created according to the
#                 distance between the elements of the molecule.
#
# usecube       - You can choose to use the .vtk+.xyz input files (usecube=False) or the .cube file (usecube=True) by setting
#                 this variable.
#
```

In-depth analysis of *C2H4_2D.py*

```
# Set the variable below as DEFAULT values

inputdata = '/home/user/inputdata.txt'
outdir    = '/home/user/output/'

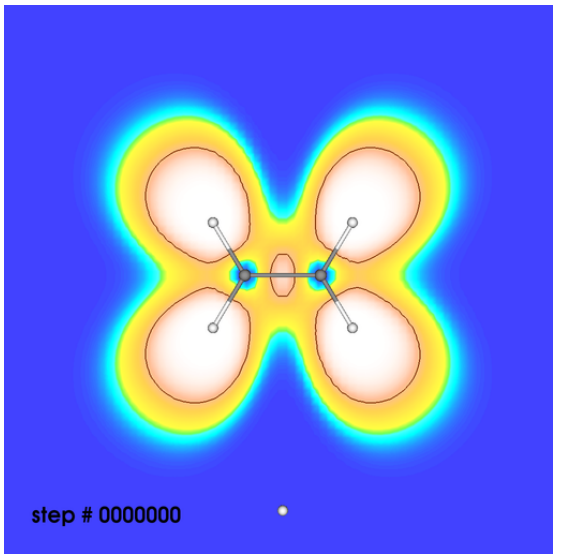
elfvtk    = 'elf_rs.vtk'
geometry  = 'geometry.xyz'
elfcube   = 'elf_rs.cube'

bonds     = False
usecube   = False
```

Default input file and output directory. Used when option 1 for the input specification is selected

Names of the input files. Note that the variables they contain must have specific names

Set to “True” if you want to plot the molecular bonds or if you want to use the .cube input file instead of the default .vtk file



In-depth analysis of *C2H4_2D.py*

```
##############################################################################
# !!!!!!!!! You are not supposed to modify what is below this point !!!!!!!!!! #
##############################################################################

################################################
# Setting up Python interpreter, if necessary #
################################################
import sys

# Detect if running in CLI ('visit -cli') and if not check Python version
isCLI = True
if 'VisItException' not in dir():
    if sys.version_info[1] < 5:
        print "\nRequires Python version 2.5 or greater\n"
        sys.exit()
    isCLI=False

# Import visit into a standard Python interpreter, if required (get error
# if run Launch() from CLI). Launch() is required to start the Viewer and
# import the rest of the API.
if not isCLI:
    print
    print "Running VisIt's script from Python interpreter"
    print
    print "Insert path to the 'site-packages' installation directory"
    visitpath = raw_input("E.g: /home/user/VisIt/current/linux-x86_64/lib/site-packages \n")
    winornot = raw_input("Do you want to use a plotting window? (y/n) \n")
    sys.path.append(visitpath)
    from visit import *
    if (winornot=="y"):
        Launch()
    else:
        LaunchNowin()
```

We need some system calls

Checking if using python or VisIt's CLI

Better if Python 2.7 because of compatibility

We must provide the path to VisIt's installation

Note: these lines work only with a Python interpreter

In-depth analysis of *C2H4_2D.py*

```
#################################
# Setting up python and inputs #
#################################
# Some useful modules
import math, time
import glob, os, shutil
import subprocess
from visit_utils import *

start_script = time.time()

# Setting initial variables or using default input
subprocess.call(["clear", ""])

print " *** Welcome to C2H4_2D.py, a visualization script for VisIt ***"
print ""
print "Description: This script creates a set of 2D plots of the scatter"
print "             of a proton off a molecule of C2H4."
print ""
print "Enter your choice:"
print "1- Use default value for the setup variables"
print "2- Specify the location of the file containing the list of input"
print "3- Specify single inputs"
answer = raw_input("\n")
```

Mathematical and time-related functions

Used for filename pattern matching, checking for paths and file operations

Access to some commands of the linux terminal

Used for the movie creation

Welcome screen

Input from keybord saved in variable “answer”

In-depth analysis of *C2H4_2D.py*

```
if (answer=="1"):
    fileex = os.path.exists(inputdata)
    if (fileex==False):
        print "File do not exists!"
        quit()
    with open(inputdata, 'r') as infile:
        indata = infile.read().split('\n')
    outex = os.path.exists(outdir)
    if (outex==False):
        print "Directory does not exists!"
        quit()
    limits    = [-5,5,-5,5]
    savename  = 'mol2D'
    moviename = ""
    subprocess.call(["clear", ""])
```

Option 1:

- Using default values for the location of the “inputdata.txt” file and for the other variables
- *indata* is a vector containing the list of input files
- Limits of the plot and output name of the images can be changed here
- By default, the movie is not created

Warning: the script will likely crash if you choose this option without setting the default variables first.

This option is useful if you need to run tests or multiple visualizations of the same set of data and you don't want to specify the location of the input/output each time.

In-depth analysis of *C2H4_2D.py*

```
elif (answer=="2"):
    print "Enter full path to the file containing the list of input:"
    inputdata = raw_input("(e.g: /home/user/inputdata.txt)\n")
    fileex = os.path.exists(inputdata)
    if (fileex==False):
        print "File does not exists!"
        quit()
    with open(inputdata, 'r') as infile:
        indata = infile.read().split('\n')
    outdir = raw_input("Enter output directory: (press Enter for current directory)\n")
    if (len(outdir)==0):
        outdir = subprocess.check_output(["pwd"]).rstrip("\n")+"/"
    elif (outdir[len(outdir)-1]!="/"):
        outdir=outdir+"/"
    outex = os.path.exists(outdir)
    if (outex==False):
        print "Directory does not exists!"
        quit()
    print "Enter plot limits separated by spaces: (xmin xmax ymin ymax)"
    limits = raw_input("Suggested values for this plot are -5 5 -5 5\n").split(" ")
    if (len(limits)!=4):
        limits = [-5,5,-5,5]
    savename = raw_input("Enter name for output images:\n")
    if (len(savename)==0):
        savename = 'mol2D'
    print "Enter name for output movie with extension: (press Enter to skip movie creation)"
    moviename = raw_input("You can choose between .wmv, .mpg and .mov file formats\n")
    if (len(moviename)!=0):
        moviespeed = raw_input("Set movie speed (1 fast - 100 slow)\n")
    subprocess.call(["clear", ""])
```

Option 2:

Still requires a “inputdata.txt” file

Selecting current directory

Default values if wrong input

Plot limits separated by single space

Default value if empty input

In-depth analysis of *C2H4_2D.py*

```
elif (answer=="3"):
    inputdir = raw_input("Enter full path to the directory containing the input timesteps:\n")
    if (inputdir[len(inputdir)-1]!="/"):
        inputdir=inputdir+"/"
    inputstart = raw_input("Enter the name of the first input (e.g. td.0000000/):\n")
    if (inputstart[len(inputstart)-1]!="/"):
        inputstart=inputstart+"/"
    inputend = raw_input("Enter the name of the last input (e.g. td.0030000/):\n")
    if (inputend[len(inputend)-1]!="/"):
        inputend=inputend+"/"
    print "Note that we assume that each input directory contains"
    print "a elf_rs.vtk, a geometry.xyz and a elf_rs.cube file"
    inputstride = raw_input("Enter the timestep distance between inputs (e.g. 15):\n")
    if (int(inputstride)==0):
        totinput=1
    else:
        totinput=(int( inputend[len(inputend)-8:len(inputend)-1] )-int( inputstart[len(inputstart)-8:len(inputstart)-1] ))/int
(inputstride) +1
    indata = [None]*(totinput)
    outdir = raw_input("Enter output directory: (press Enter for current directory)\n")
    if (len(outdir)==0):
        outdir = subprocess.check_output(["pwd"]).rstrip("\n")+"/"
    elif (outdir[len(outdir)-1]!="/"):
        outdir=outdir+"/"
    outex = os.path.exists(outdir)
    if (outex==False):
        print "Directory does not exists!"
        quit()
```

Option 3:

Checking if we need to add a final “/”

Counting the inputs and preparing the *indata* vector

In-depth analysis of *C2H4_2D.py*

```
    print "Enter plot limits separated by spaces: (xmin xmax ymin ymax)"
    limits = raw_input("Suggested values for this plot are -5 5 -5 5\n").split(" ")
    if (len(limits)!=4):
        limits = [-5,5,-5,5]
    savename = raw_input("Enter name for output images:\n")
    if (len(savename)==0):
        savename = 'mol2D'
    print "Enter name for output movie with extension: (press Enter to skip movie creation)"
    print "Note that you need an encoder like ffmpeg on this machine in order to be able to create a movie"
    moviename = raw_input("You can choose between .wmv, .mpg and .mov file formats\n")
    if (len(moviename)!=0):
        moviespeed = raw_input("Set movie speed (1 fast - 100 slow)\n")
    subprocess.call(["clear", ""])
    for i in range(0,totinput):
        step=int( inputstart[len(inputstart)-8:len(inputstart)-1] )+int(inputstride)*i
        indata[i]=inputdir + inputstart[0:len(inputend)-8] +"{0:07d}".format(step)+"/"
else:
    print "Value not recognized."
    quit()
```

Plot's limits and output name (same as option 2)

Creating the list of input in the vector *indata* for option 3

Note: it is possible to create a movie with old images in the output directory by having a *indata* vector empty. This can be done with an empty “inputdata.txt” file or by specifying input files that are not available (e.g. td.9999999/).

In-depth analysis of *C2H4_2D.py*

```
# Checking if current directory coincides with output directory
changedir=False
currentpath = subprocess.check_output(["pwd"]).rstrip("\n")+"/"
if (currentpath!=outdir):
    changedir=True

# Checking if input files exist
indata = filter(None, indata)
for i in range(0,len(indata)):
    direx = os.path.exists(indata[i])
    if (direx==False):
        print "Warning! Input directory ",indata[i]," DOES NOT exist! Removing it from input list...\n"
        indata[i]=None

# Removing empty lines
indata = filter(None, indata)

# Check if there are still input files available
if (len(indata)==0):
    print "Total number of input files is 0!"
    print "Please check the paths and the name of the inputs"
    print ""
    print "Note that the script will continue in case you want to produce a video with old images"
    press = raw_input("Press enter to continue or type \"q\" to quit\n")
    if (press=='q'):
        quit()
```

The script removes from vector *indata* all the input files that can not be found in the specified directory

Removing all empty lines from *indata*

Even if *indata* is empty, you can continue in order to create a movie with old images

In-depth analysis of *C2H4_2D.py*

```
# Summary of setup variables
print "Summary of the setup variables:\n"
print "-Total number of input: ", len(indata)
if (len(indata)!=0):
    print "-First input: ", indata[0]
if (len(indata)!=0):
    print "-Last input:  ", indata[len(indata)-1]
print "-Output directory: ", outdir
print "-Image limits: ",limits[0],"< X <", limits[1]
print "               ",limits[2],"< Y <", limits[3]
print "-Images name: ", savename+"####.png"
if (len(moviename)==0):
    print "-Movie file will not be created"
else:
    print "-Movie file "+moviename+" will be created"
    print "-Movie speed: ", moviespeed
print ""
press = raw_input("Press enter to continue or type \"q\" to quit\n")
if (press=='q'):
    quit()
subprocess.call(["clear", ""])
```

Summary

In-depth analysis of *C2H4_2D.py*

```
########################
# We now start plotting #
########################
# Creating color table
ct = ((60,60,255,0.), (0,255,255,0.1), (65,255,255,0.16), (111,255,79,0.19), (255,255,60,0.26), (255,208,74,0.65), (255,187,14
6,0.85), (255,255,255,1))
ccpl = ColorControlPointList()
for pt in ct:
    p = ColorControlPoint()
    p.colors = (pt[0], pt[1], pt[2], 255)
    p.position = pt[3]
    ccpl.AddControlPoints(p)
AddColorTable("elf", ccpl)
cmapname="elf"
if (answer!="1"):
    print "Current color table is "+cmapname
    cmans = raw_input("Do you want to change it? (y/n)\n")
    if (cmans=="y"):
        print "The available color tables are:"
        for ctnames in ColorTableNames():
            print "- "+ctnames
        print ""
        print "Note: if you want to define a new color table, please refer to"
        print "      the VisIt manual."
        cmapname = raw_input("Enter the new color table:\n")
```

Defining a new color table

VisIt uses 3 numbers (RGB) plus opacity. Here, we are using the 4th number as position for the control point

All opacities are equal to 255 (e.g. no transparency)

Saving the new color table as “elf”

You can refer to websites like *www.rapidtables.com/web/color/RGB_Color.html* for the RGB value of colors

Exercise: change the color table to any other set of 5 equally spaced colors

In-depth analysis of *C2H4_2D.py*

```
# Setting annotations
a = AnnotationAttributes()
a.userInfoFlag = 0
a.databaseInfoFlag = 0
a.axes2D.visible = 0
SetAnnotationAttributes(a)

# Setting View
wa = GetView2D()
wa.windowCoords = (float(limits[0]),float(limits[1]),float(limits[2]),float(limits[3]))
wa.viewportCoords = (0, 1, 0, 1)
wa.fullFrameActivationMode = wa.On
SetView2D(wa)
SetViewExtentsType("actual")
ToggleLockViewMode()
ToggleMaintainViewMode()

# Setting saving options
s = SaveWindowAttributes()
s.format = s.PNG
s.fileName = savename
s.width, s.height = 1024,1024
s.resConstraint = 0
if (changedir==True):
   s.outputToCurrentDirectory = 0
   s.outputDirectory = outdir
else:
   s.outputToCurrentDirectory = 1
SetSaveWindowAttributes(s)
```

Window annotation:
0 = no 1 = yes

Limits of the view
Setting the full frame
And locking the view to current values

Each image will be a .png with 1024x1024 pixels

Several other formats (e.g. jpeg, bmp, tiff, ps) are available

In-depth analysis of *C2H4_2D.py*

```
# Creating a label for future use
l = CreateAnnotationObject("Text2D")
l.text = "step # 0000000"
l.position = (0.05, 0.05)
l.height = 0.03
l.fontBold = 1

# Cycling on all input files
imgtstart = time.time()
#for i in range(0,1):     # Use only the first input for tests
for i in range(0,len(indata)):
    activepath = indata[i]
    if (activepath[len(activepath)-1]!="/"):
        activepath = activepath+"/"
    invtk = activepath+elfvtk
    inxyz = activepath+geometry
    incube = activepath+elfcube
    step = activepath[len(activepath)-8:len(activepath)-1]

# Adding a label for the time step
    l = GetAnnotationObject("Text2D1")
    l.text = "step # "+step
```

Creating a new object

x, y between 0 and 1

Other objects available:

- Text3D
- TimeSlider
- Image
- Line2D (and arrows)

Loop on all entries of *indata*

Recovering the full path to the input files (including name of the database)

Retrieving the information of the 1st label

Exercise: create a new label with some static text (e.g. a Title) using Italic and Times New Roman fonts.

In-depth analysis of *C2H4_2D.py*

```
# Opening Database
    if (usecube==False):
        OpenDatabase(invtk)
        elfvar="Re"
    else:
        OpenMDServer()
        DBopt = GetDefaultFileOpenOptions("Cube")
        DBopt["ExtendVolumeByOneCell"] = 0
        SetDefaultFileOpenOptions("Cube", DBopt)
        OpenDatabase(incube, 0, "Cube")
        elfvar="electron_density"
```

Opening default .vtk database

Explicitly opens a metadata server. This allows to provide command line options that influences how the metadata server will run.

Setting “special” file options when opening the .cube
Without these lines the .cube electron_density looks stretched along a line.

After changing the file options, we can open the file safely.

In-depth analysis of *C2H4_2D.py*

```
# Adding Psedocolor of the elf
    AddPlot("Pseudocolor", elfvar)

# Adding a slice operator to this plot
    AddOperator("Slice", 0)
    sli = SliceAttributes()
    sli.originType = sli.Point
    sli.originPoint = (0, 0, 0.075)
    sli.axisType = sli.ZAxis
    SetOperatorOptions(sli)

# Pseudocolor plot option
    p = PseudocolorAttributes()
    p.colorTableName = cmapname
    p.legendFlag = 0
    p.minFlag,p.min = 1, 0.
    p.maxFlag,p.max = 1, 1.
    p.scaling = 0
    SetPlotOptions(p)
```

1st plot: electron localization function

Slicing current plot:

- Specifying the slice origin according to the position of the molecule
- Along Z-axis

Pseudocolor options:

- Specifying color table
- No legend
- Limiting values between 0 and 1
- Linear scaling

Setting the new values. Note that this command does not depend on the plot type.

In-depth analysis of *C2H4_2D.py*

```
# Adding contour at 0.8
    AddPlot("Contour", elfvar)

# Contour plot option
    c = ContourAttributes()
    c.colorType = c.ColorBySingleColor
    c.singleColor = (118, 42, 0, 255)
    c.contourMethod = 1
    c.contourValue = (0.8)
    c.lineWidth= 1
    c.legendFlag = 0
    SetPlotOptions(c)

# Adding a slice operator to this plot
    AddOperator("Slice", 0)
    sli = SliceAttributes()
    sli.originType = sli.Point
    sli.originPoint = (0, 0, 0.075)
    sli.axisType = sli.ZAxis
    SetOperatorOptions(sli)
```

2nd plot: contour at 0.8

Contour:

- Using a single contour
- With user-selected color (with full opacity)
- Contours chosen by value
- No legend

We slice the current plot with the same options used before

Exercise: add 2 contours (blue and purple) at 0.9 and 0.5

In-depth analysis of *C2H4_2D.py*

```
# Choosing the Database and adding the plot
    if (usecube==False):
        # Opening .xyz Database
        OpenDatabase(inxyz)

# Adding molecule and projecting it to the plane
    AddPlot("Pseudocolor", "element")
    mol = PseudocolorAttributes()
    mol.colorTableName = "cpk_jmol"
    mol.minFlag,mol.min = 1, -1.
    mol.maxFlag,mol.max = 1, 118.
    mol.legendFlag = 0
    mol.pointSizePixels = 25
    mol.pointType = mol.Sphere
    mol.lineType = mol.Line
    mol.lineWidth = 1
    mol.renderSurfaces = 1
    mol.renderPoints = 1
    mol.smoothingLevel = 2
    SetPlotOptions(mol)

    AddOperator("Project", 0)
    pro = ProjectAttributes()
    SetOperatorOptions(pro)
```

3rd plot: elements

Opening default .xyz database

Plotting the molecule using a Pseudocolor plot

Reproducing the color table for molecules

Using spheres for the elements of the molecule

Important:
All the plots must live in the same space (either 2D or 3D), therefore we must use a projection

Exercise: plot a Molecule instead of a Pseudocolor plot

In-depth analysis of *C2H4_2D.py*

```
# Creating bonds
    if (bonds==True):
        AddOperator("CreateBonds", 0)

# Drawing
    DrawPlots()

#  Saving the image
    SaveWindow()

# Delay before cleaning
#    time.sleep(1)
```

Saving image

Adding the molecular bonds, if necessary

Now that all the plots are ready, we can actually draw them

Images will be saved using the SaveWindowAttributes() defined before

VisIt will save the images using the base name and (by default) 4 digits.
If the file name is already present, VisIt will issue a warning and increase the digits to the first available file name.

In-depth analysis of *C2H4_2D.py*

```
# Clear and close
    DeleteAllPlots()
    if (usecube==False):
        CloseDatabase(inxyz)
        CloseDatabase(invtk)
    else:
        CloseDatabase(incube)

# Estimating remaining time
    print "Time left (sec): ",((time.time()-imgtstart)/(i+1))*(len(indata)-(i+1))

# End of the loop over the input files
```

Cleaning windows and closing databases

Estimate of the remaining time for the completion of all the images. Based on the time required to complete “i” images and on the total number of images

In-depth analysis of *C2H4_2D.py*

```
imgtend = time.time()
subprocess.call(["clear", ""])
print "Creation of the images completed."

# Movie creation
if (len(moviename)!=0):
    vidtstart= time.time()
    input_pattern = outdir+savename+"%04d.png"
    output_pattern = outdir+moviename
    encoding.encode(input_pattern,output_pattern,fdup=int(moviespeed))
    vidtend = time.time()
```

Movie creation

Using images with “correct” name available in the output directory

Alternatively, you can use the command `visit -movie` to produce a movie.
See *www.visitusers.org/index.php?title=Making_Movies* for details and options.

Exercise: make a movie with old images using the script

In-depth analysis of *C2H4_2D.py*

Time statistics

```
# Reporting working time
print "\n"
print " *** Time statistics ***"
if (len(indata)!=0):
    print "Images creation time:"
    print("--- %s seconds ---" % (imgtend - imgtstart))
    print( "On average %s seconds per image" % ((imgtend - imgtstart)/len(indata)) )
    print ""
print "Video creation time:"
if (len(moviename)!=0):
    print("--- %s seconds ---" % (vidtend - vidtstart))
else:
    print"--- Not Available ---"
print ""
print "Script total execution time:"
print("--- %s seconds ---" % (time.time() - start_script))
print ""

quit()
```

Features from *C2H4_3D.py*

Camera movements using control points:

```
def rotatecamera():
    # Create the control points for the views.
    c0 = View3DAttributes()
    c0.viewNormal = (0, 0, 1)
    c0.focus = (0, 0, 0)
    c0.viewUp = (0, 1, 0)
    c0.viewAngle = 30
    c0.parallelScale = 12.
    c0.nearPlane = -24.
    c0.farPlane = 24.
    c0.imageZoom = float(zoom)
    c0.perspective = 1

    c1 = View3DAttributes()
    c1.viewNormal = (-1, 0, 0)
    c1.focus = (0, 0, 0)
    c1.viewUp = (0, 1, 0)
    c1.viewAngle = 30
    c1.parallelScale = 12.
    c1.nearPlane = -24.
    c1.farPlane = 24.
    c1.imageZoom = float(zoom)
    c1.perspective = 1
```

Definition of control points that will be used for camera movements.

Note:
You can use the VisIt's GUI to identify the attributes of “interesting” control points.

Features from *C2H4_3D.py*

Camera movements using control points:

```
c2 = View3DAttributes()
c2.viewNormal = (0, 0, -1)
c2.focus = (0, 0, 0)
c2.viewUp = (0, 1, 0)
c2.viewAngle = 30
c2.parallelScale = 12.
c2.nearPlane = -24.
c2.farPlane = 24.
c2.imageZoom = float(zoom)
c2.perspective = 1

c3 = View3DAttributes()
c3.viewNormal = (1, 0, 0)
c3.focus = (0, 0, 0)
c3.viewUp = (0, 1, 0)
c3.viewAngle = 30
c3.parallelScale = 12.
c3.nearPlane = -24.
c3.farPlane = 24.
c3.imageZoom = float(zoom)
c3.perspective = 1

c4 = c0
```

Additional control points

Note that the last control point coincides with the first one in order to close the camera rotation

Features from *C2H4_3D.py*

Camera movements using control points:

```
# Create a tuple of camera values and x values. The x values are weights
# that help to determine where in [0,1] the control points occur.
cpts = (c0, c1, c2, c3, c4)
x=[]
for i in range(5):
    x = x + [float(i) / float(4)]

# Animate the camera. Note that we use the new built-in EvalCubicSpline
# function which takes a t value from [0,1] a tuple of t values and a tuple
# of control points. In this case, the control points are View3DAttributes
# objects that we are using to animate the camera but they can be any object
# that supports +, * operators.
nsteps = 100
for i in range(nsteps):
    t = float(i) / float(nsteps - 1)
    c = EvalCubicSpline(t, x, cpts)
    SetView3D(c)
    if (i!=nsteps-1):
        SaveWindow()
```

List of control points

Number of control points

Number of control points -1

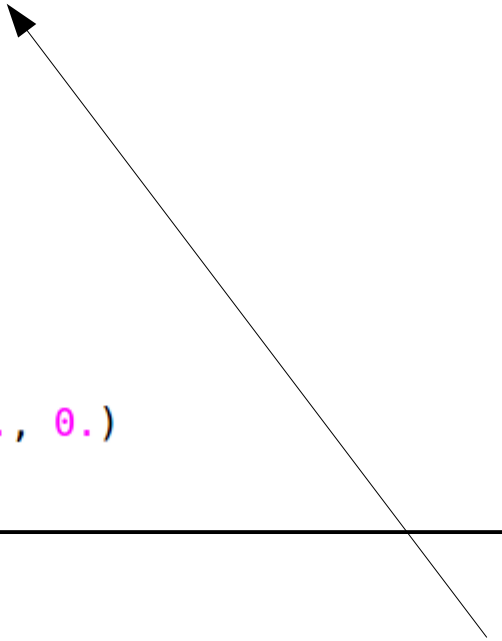
Number of steps to complete the camera movement

Exercise: add 2 control points for camera movements zooming in and out on the molecule

Features from *C2H4_3D.py*

Camera rotation using functions:

```
# Keep rotating the camera during scattering
    if (keeprot==True):
        kr = View3DAttributes()
        x = math.sin(2.*math.pi*(float(i)/float(len(indata)-1)))
        z = math.cos(2.*math.pi*(float(i)/float(len(indata)-1)))
        kr.viewNormal = (-x, 0, z)
        kr.focus = (0, 0, 0)
        kr.viewUp = (0, 1, 0)
        kr.viewAngle = 30
        kr.parallelScale = 12.
        kr.nearPlane = -24.
        kr.farPlane = 24.
        kr.imageZoom = float(zoom)
        kr.perspective = 1
        kr.centerOfRotation = (0., 0., 0.)
        SetView3D(kr)
```



You can also use functions to define camera positions.
Note that, in the definition of viewNormal, $x^2+y^2+z^2$ must be equal to 1

Features from *C2H4_3D.py*

Volume plot opacity:

```
    elif (plottype=='2'):
        AddPlot("Volume", elfvar)
        ray = VolumeAttributes()
        ray.legendFlag = 0
        ray.lightingFlag = 1
        ray.colorControlPoints.GetControlPoints(0).colors = (0, 255, 255, 255)
        ray.colorControlPoints.GetControlPoints(0).position = 0.436155
        ray.colorControlPoints.GetControlPoints(1).colors = (255, 102, 0, 255)
        ray.colorControlPoints.GetControlPoints(1).position = 0.536585
        ray.colorControlPoints.GetControlPoints(2).colors = (255, 255, 0, 255)
        ray.colorControlPoints.GetControlPoints(2).position = 0.75
        ray.colorControlPoints.GetControlPoints(3).colors = (255, 0, 0, 255)
        ray.colorControlPoints.GetControlPoints(3).position = 1
        ray.opacityAttenuation = 0.18
        ray.opacityMode = ray.FreeformMode
        ray.resampleTarget = 50000
        ray.freeformOpacity = (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 2, 4, 5, 7, 8, 11, 14, 19, 22, 23, 26, 28, 28, 29, 31, 32, 34, 34, 35, 37, 38, 40, 41, 43, 43, 43, 43, 44, 46, 47, 49,
 49, 49, 49, 49, 49, 49, 49, 50, 50, 50, 50, 50, 50, 50, 50, 50, 50, 49, 46, 43, 41, 40, 38, 37, 35, 35, 34, 31, 29, 29, 28, 25
, 20, 16, 11, 8, 5, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 2, 9, 14, 20, 25, 28, 31, 34, 37, 40, 42, 46, 49, 52, 55, 56, 59, 61, 61, 62, 64, 65, 67, 68, 70, 70, 70, 70, 70, 70, 70, 70,
70, 70, 70, 70, 70, 67, 65, 65, 64, 61, 59, 58, 55, 52, 49, 46, 43, 40, 35, 32, 29, 25, 19, 14, 11, 8, 5, 2, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 8, 20, 31, 42, 49, 53, 55, 60, 65,
 70, 70, 70, 70, 70, 70, 70, 70, 70, 70, 70)
        ray.smoothData = 1
        ray.samplesPerRay = 200
        ray.rendererType = ray.RayCasting
        SetPlotOptions(ray)
```

Quite complicated expressions for the color table and transfer function in the Volume plot (with ray casting)

Exercise: change the transfer function and color in the Volume plot
(Hint: first try using the GUI)

Features from azulene.*py*

Multiple windows in a single image:

```
# This script will create 4 different windows (plus a 5th window used for calculations, but then destroyed).
# Therefore, we start by defining the saving options for these images.
# We want to obtain something like:
#  _____________
# |_____Win4____|
# |_____Win3____|
# |      |      |
# |__Win1_|_Win2_|
#
# Setting saving options
s = SaveWindowAttributes()
s.format = s.PNG
s.fileName = savename
s.width, s.height = 2048,1536
s.resConstraint = 0
if (changedir==True):
   s.outputToCurrentDirectory = 0
   s.outputDirectory = outdir
else:
   s.outputToCurrentDirectory = 1
s.advancedMultiWindowSave = 1
s.subWindowAtts.win1.size = (1024,1024)
s.subWindowAtts.win1.position = (0,0)
s.subWindowAtts.win2.size = (1024,1024)
s.subWindowAtts.win2.position = (1024,0)
s.subWindowAtts.win3.size = (2048,256)
s.subWindowAtts.win3.position = (0,1024)
s.subWindowAtts.win4.size = (2048,256)
s.subWindowAtts.win4.position = (0,1280)
SetSaveWindowAttributes(s)
```

Exercise: switch the location of the saved windows

```
 _____________
|__Mol__|__Prop___|
|_____Slice_____|
|____Pulse Int.____|
```

Positions of the windows are always measured from the lower left corner of the window.

Features from azulene.*py*

Definition of a function:

```
# We define some functions usefull for all windows
def extension(dataset, var):     # this function creates a temporary window, calculates the extensions of a dataset and destroys the w
indow
    AddWindow()
    ReOpenDatabase(dataset)
    AddPlot("Pseudocolor", var)
    DrawPlots()
    Query("SpatialExtents")
    ext = GetQueryOutputValue()
    DeleteWindow()
    return ext
```

Defining a function with two arguments

Extracting the values from a query

Calling a function:

```
    # Retriving extention of the E-field along x dataset
    ext = extension(inefx, fldx)
```

Several (34) predefined queries are available in VisIt.
See the *VisIt User's Manual* for a complete description

Note:
we need a non-hidden plot to perform a query. Therefore, we use an additional window for all the calculations that we destroy once we are done

Exercise: define a new query, print the results of the mean, std deviation, etc. and save them into a variable

Features from azulene.*py*

Definition of new expressions:

```
DefineScalarExpression(“new_var”, “definition”)
```

E.g:

```
DefineScalarExpression(“logT”, “log10(Temperature)”)
DefineScalarExpression(“Over0”, “if(gt(value,0), 1, 0)”)
```

Exercise: define a new scalar variable using “Re” that goes from 0 to 10 with all values < 2 set to 0.

Vector expression with connected components:

```
# We define a vector field from the [x,y,z] scalar components of the electric field
expression = "{conn_cmfe(<"+inefx+":"+fldx+">,mesh), conn_cmfe(<"+inefy+":"+fldy+">,mesh), conn_cmfe(<"+inefz+":"+fldz+">,mesh)}"
DefineVectorExpression("Evec", expression)

expression2D = "{conn_cmfe(<"+inefx+":"+fldx+">,mesh), conn_cmfe(<"+inefy+":"+fldy+">,mesh), 0.}"
DefineVectorExpression("Evec2D", expression2D)
```

It's now safe to turn off your computer.

Thank you!